

TO39-D3应用设计指导

一、概述

TO39-D3是一款直插形式的数字红外热电堆芯片，用于非接触测温应用。芯片内置热电堆传感器和专用处理芯片，用户无需其他外围，直接通过 IIC 总线与传感器进行通讯读取最终测量温度。

本产品带出厂标定，适合于门禁系统、工业测温远距离测温应用。



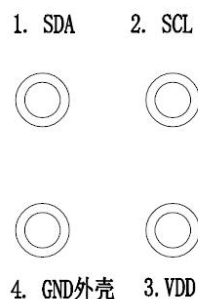
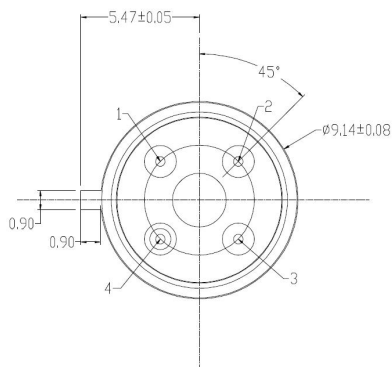
二、结构设计

1、管脚定义

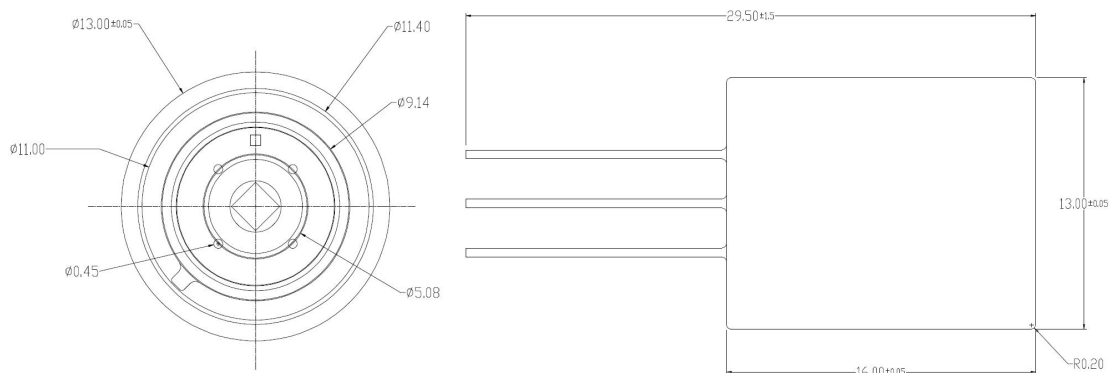
序号	功能	备注
1	SDA: IIC 总线数据线	
2	SCL: IIC 总线时钟线	
3	VDD: 电源正	
4	GND: 电源地，连接到外壳	

底视图

2、外形尺寸



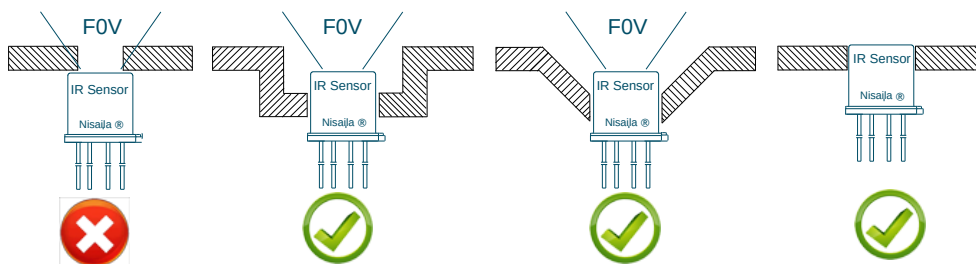
基体底视图



基体轮廓尺寸

3、结构设计要求

对于带金属光杯的型号，如果结构上允许，可以将传感器镜头突出设备外壳上表面。如果外观有限制需要将器件内置，则需要保证结构避让器件视场。



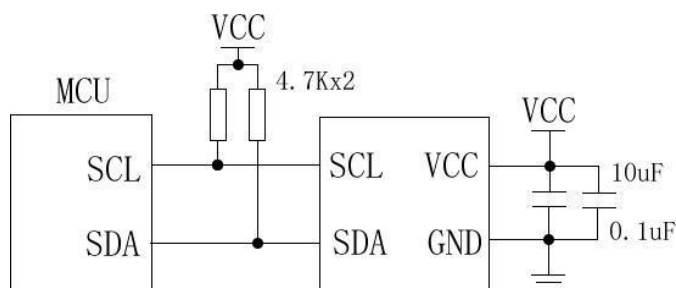
传感器本身自带红外透镜，因为滤光片对红外线有衰减，会导致测量结果偏低，如非必要，不要再增加其他滤光片。

4、热设计要求

环境温度对测温效果影响较大，尽量保证传感器处于一个稳定的环境温度下，避免热源对其影响。因此要远离高发热器件，PCB上传感器四周尽可能做开槽设计，将传感器置于孤岛。

三、硬件设计

器件引脚包含电源、IIC 总线共 4 个管脚，供电电压允许范围为 2.3~3.6V。传感器本身功耗很低，电源地之间使用一颗 0.1uF 电容即可，如果传感器离供电部分比较远，可以考虑增加一个 10uF 电容，以保证电源稳定降低噪声。

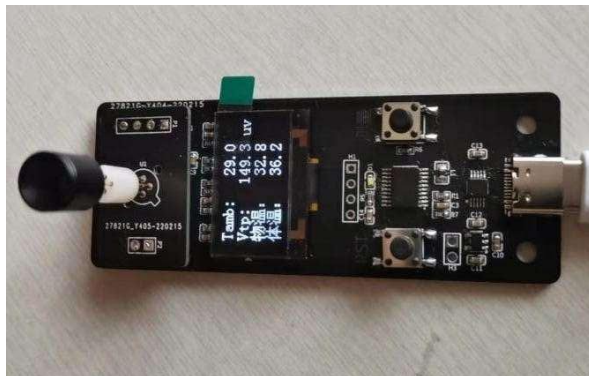


四、程序设计

传感器采用 IIC 接口，器件上电后需要等待 2mS 以上，待器件复位并完成初始化操作即可以直接读取测量温度结果。

五、附录 1：demo 板使用教程

为配合合作伙伴快速上手，本司开发了一块 demo 板，用于对各型号传感器进行评估测试。测量结果会在屏幕上显示，并可以通过 USB 转串口打印到 PC 端串口调试软件。



实物照片

说明：

1、demo 板通过 Type-C 插座供电，demo 板供电电压为 5V。使用时将 demo 板通过 Type-c 连接至 PC 端，即可从屏幕上查看结果。如需串口显示数据，需安装沁恒 CH340 驱动程序然后使用串口调试软件进行操作，串口通信设置为：

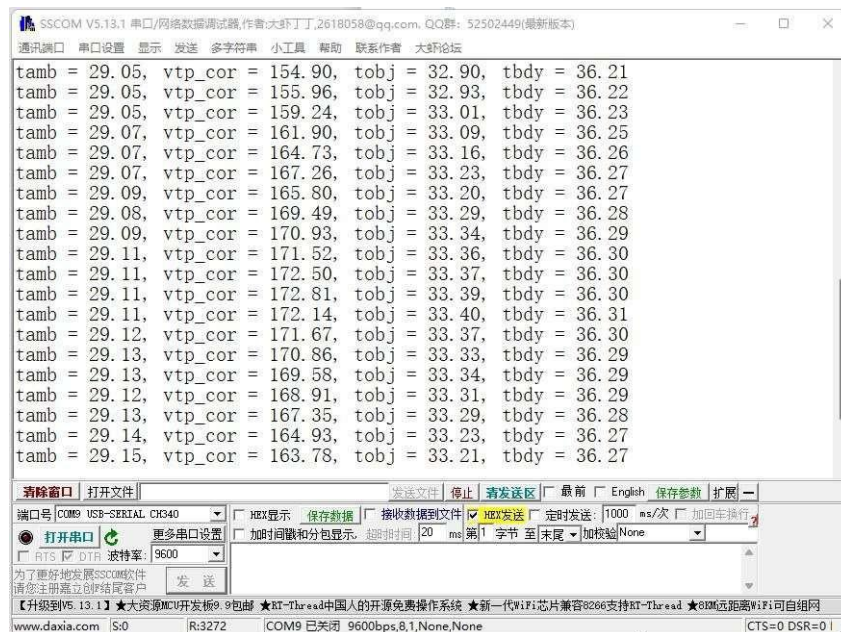
波特率：9600

数据位：8

停止位：1

无奇偶校验

2、demo 板屏幕显示从上到下依次是：环境温度（tamb）、传感器输出电压（vtp_cor）、物体（表面）温度（tobj）、人体温度（tbody）。上述数值温度单位为℃，电压单位为 uV。其中前三个值为器件原始输出，体温值使用我司算法根据原始值计算得到，仅供参考，用户可以根据需要使用自家体温算法来处理数据。



串口显示效果

3、左侧按键为复位，右侧按键可以控制程序启动、暂停。


```

        if (run) {
            Trs_Read ();
            OLED_Refresh();
        }
        delay_ms(20);
    }
}

```

```

////////////////////////////////////
// trs55d.h

```

////

```
#ifndef _TRS_H_
```

```
#define _TRS_H_
```

```
#include <stdint.h>
```

```
#define NORMAL_Tobj_MSB_R      0x10
```

```
#define NORMAL_Tobj_CSB_R 0x11
```

```
#define NORMAL_Tobj_LSB_R 0x12
```

```
#define NORMAL_TEMP_MSB_R      0x16
```

```
#define NORMAL_TEMP_CSB_R      0x17
```

```
#define NORMAL_TEMP_LSB_R      0x18
```

```
#define NORMAL DATA1 MSB R 0x19
```

```
#define NORMAL_DATA1_CSB_R      0x1A
```

```
#define NORMAL_DATA1_LSB_R      0x1B
```

```
#define NORMAL_DATA2_MSB_R      0x1C
```

```
#define NORMAL_DATA2_CSB_R      0x1D
```

```
#define NORMAL_DATA2_LSB_R      0x1E
```

```
#define RAW_DATA1_MSB_R      0x22
```

```
#define RAW_DATA1_CSB_R 0x23
```

```
#define RAW_DATA1_LSB_R 0x24
```

```
#define RAW_DATA2_MSB_R      0x25
```

```
#define RAW_DATA2_CSB_R 0x26
```

```
#define RAW_DATA2_LSB_R 0x27
```

```
#define RAW_TEMP_MSB_R 0x28
```

```
#define RAW_TEMP_CSB_R      0x29
```

```
#define RAW_TEMP_LSB_R 0x2A
```

```
#define TRS_ADDR_AD    (0x7F << 1) //0xfe
```

```
#define _STATUS_DRDY_ 0x01
```

```
void Trs_Read(void);
```

```
#endif
```

////////////////////////////////////

```
// trs55d.c
```

////

```
#include <stdio.h>
#include "drv_i2c.h"
#include "trs55d.h"
#include "body_temp.h"

extern uint8_t uartSendString(const char* buf);
extern void OLED_ShowString(uint8_t x,uint8_t y, const char *chr,uint8_t size1,uint8_t mode);
extern void OLED_ShowBNum(uint8_t x,uint8_t y,float num,uint8_t len,uint8_t size2,uint8_t
mode);

uint8_t TRS55D_IIC_Read(uint8_t addr_dev, uint8_t addr_reg, uint8_t *buf, uint16_t count)
{
    uint8_t ret;
    uint8_t ackflag;
    uint16_t i = 0;
    drv_i2c_start();
    drv_i2c_select_dev(addr_dev,DRV_I2C_OPWR);
    drv_i2c_writebyte(addr_reg);
    drv_i2c_start();
    drv_i2c_select_dev(addr_dev,DRV_I2C_OPRD);
    for(i = 0; i < count; i++) {
        ackflag = (i < (count-1)) ? 1:0; buf[i]
        = drv_i2c_readbyte(ackflag);
    }
    drv_i2c_stop();
    return ret;
}

void TRS55D_IIC_Write(uint8_t addr_dev, uint8_t addr_reg, uint8_t *buf, uint16_t count)
{
    uint16_t i = 0;
    drv_i2c_start();
    drv_i2c_select_dev(addr_dev,DRV_I2C_OPWR);
    drv_i2c_writebyte(addr_reg);
    for (i = 0; i < count; i++)
        { drv_i2c_writebyte(buf[i]);
        }
    drv_i2c_stop();
}

typedef union{
    int16_t i16;
    uint16_t u16;
    struct {
        uint8_t u8l;
```

```

        uint8_t u8h;
    }un;
}uu16_t;

typedef union{
    int32_t i32;
    struct {
        uint8_t u8b0;
        uint8_t u8b1;
        uint8_t u8b2;
        uint8_t u8b3;
    }un;
}uu32_t;

static char buffer[264];
uu32_t obj,tmp_3, vtp_cor;
float tamb_temp ,vtp_cor_uv ,Tobj_temp ;

#include "FW_TRS_CAL.h"
uint16_t t_objx10_u16;
int16_t Tambx10_u16;
void Trs_Read(void)////////////////////////////////////
{

    uint8_t rbuf[4];
    uint8_t raddr, rdat;
    uint8_t waddr = 0x0, wdat = 0x0;
    int timeout=0;

    waddr = 0x30;//CMD
        wdat = 0x09;//0x0000 1 FSM 001 单通道连续转换
    TRS55D_IIC_Write(TRS_ADDR_AD, waddr, &wdat,1); //start conversion 模式使能
    delay_ms(10);
    raddr = 0x03;//原始数据地址
    do {
        TRS55D_IIC_Read (TRS_ADDR_AD, raddr, &rdat,1);
    } while(((rdat == 0xFF) || (!(rdat & 0x30))) && timeout++ < 100); //全真为真
                                                                    //flag置1 结果为真 取反为假跳出循环

    raddr = 0x02;//经过校准后的数据地址
    do {
        TRS55D_IIC_Read (TRS_ADDR_AD, raddr, &rdat,1);
    } while(((rdat == 0xFF) || (!(rdat & 0x0B))) && timeout++ < 200);
                                                                    //flag置1 结果为真 取反为假跳出循环

    delay_ms(50);

    raddr = NORMAL_DATA1_MSB_R;//0x19 通道1校准后adc数据
    TRS55D_IIC_Read (TRS_ADDR_AD, raddr, &rbuf[0],3);

    vtp_cor.un.u8b2 = rbuf[0];
    vtp_cor.un.u8b1 = rbuf[1];
    vtp_cor.un.u8b0 = rbuf[2];
    if (vtp_cor.un.u8b2 & 0x80) {

```

```
vtp_cor.un.u8b3 = 0xFF;
}else {
    vtp_cor.un.u8b3 = 0x00;
}
float vt_cor = 524288;
vtp_cor_uv = (float)vtp_cor.i32/vt_cor;//vtp_cor_uv=DATA1/2^19 (±16mV) =vtp_cor.i32/vt_cor
vtp_cor_uv *= 1000.0;

raddr = NORMAL_TEMP_MSB_R;//0x16环境校准后温度
TRS55D_IIC_Read (TRS_ADDR_AD, raddr, &rbuf[0],3);

tmp_3.un.u8b2 = rbuf[0];
tmp_3.un.u8b1 = rbuf[1];
tmp_3.un.u8b0 = rbuf[2];
if (tmp_3.un.u8b2 & 0x80) {
    tmp_3.un.u8b3 = 0xFF;
}else {
    tmp_3.un.u8b3 = 0x00;
}

raddr = NORMAL_Tobj_MSB_R;//0x10 dsp处理数据据
TRS55D_IIC_Read (TRS_ADDR_AD, raddr, &rbuf[0],3);

obj.un.u8b2 = rbuf[0];
obj.un.u8b1 = rbuf[1];
obj.un.u8b0 = rbuf[2];
if (obj.un.u8b2 & 0x80) {
    obj.un.u8b3 = 0xFF;
}else {
    obj.un.u8b3 = 0x00;
}

Cal_get_temp(tmp_3.i32,1000,obj.i32,1000,&Tambx10_u16,&t_objx10_u16);
Tobj_temp = (float)t_objx10_u16/10;
tamb_temp = (float)Tambx10_u16/10;
sprintf(buffer, "vtp_cor = %7.2fμV, Tobj = %5.2f℃ , tamb = %5.2f℃\r\n"

, vtp_cor_uv, Tobj_temp, tamb_temp);

uartSendString(buffer);
```

七、其他建议

1. 超过绝对最大额定值的压力和静电放电可能会损坏传感器，请采取适当的处理预防措施。不要让传感器接触腐蚀性清洁剂，光窗可以用酒精和棉签擦净。
2. 传感器应储存在密封良好的环境中，并注意防潮、防湿、防震、防尘、防静电。
3. 将本公司产品应用到一些特殊情况(即不当操作会直接影响半导体寿命或导致物理损害)之前，应事先咨询深圳尼赛拉传感器有限公司及代表。本公司不会为未经准许的操作承担任何责任。

版权所有 © 深圳尼赛拉传感器有限公司。

深圳尼赛拉传感器有限公司保留对本文档的更改而不需另行通知的权利！